Lab 49: Source-level passes first, integration second

Topic 4: Prompt Engineering & Structured Output · Task 4.6: Design multi-instance and multi-pass review architectures

Runs on Simulator 3: Multi-Agent Research System · Self-practice

Source: SkillCertPro practice exam Part 1, Q19 (unofficial third-party question)

The question

Recent evaluations show that the final report sometimes treats the same source as both confirming and contradicting a claim, and reviewers find that unsupported claims in long source bundles are missed. The team also wants review checkpoints that can log and score before the report is generated. What should the team do?

A. Run an explicit prompt chain with source-level review passes, then run a separate integration pass over the reviewed findings.

B. Launch several identical reviewers on the full source bundle and accept findings that appear most often.

C. Add confidence scores to every cited claim and route only low-confidence claims to another reviewer.

D. Send the complete source bundle to one reviewer and require a longer checklist before report generation.

Simulate it live

```
# In the browser: open Simulator 3, then Lab 49
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 49

# In a terminal, from simulators/simulator-3-multi-agent-research/
uv run research_system.py --case UC-19
```

What you should see

Reviewers get bundles smaller than the synthesis, and integration runs last.

Section reviewers get bundles smaller than the whole synthesis; the integration reviewer runs last and names conflicts and coverage gaps.

Open `simulators/simulator-3-multi-agent-research/evidence/` and find it yourself.

The answer

A. An explicit prompt chain with checkpoints you can log and score.

Treat this as explicit prompt chaining: separate API calls for focused source-level review, then pass their structured findings into a cross-source integration review. Chaining gives visible intermediate outputs, which serve as the checkpoints for logging and scoring. A large context window does not make every large review reliable: accuracy and recall degrade as tokens grow (“context rot”). Local passes reduce competing attention demands, and the integration pass focuses on consistency and conflict resolution. The cost is more calls and orchestration.

Why the others fail

B. Duplicates the same overloaded review. Voting does not fix attention dilution and provides no per-source checkpoints.

C. Self-reported confidence is uncalibrated, so confident errors slip through, and the task's shape is unchanged.

D. The same overloaded single pass, with even more competing demands added.

Check yourself

- each section-review packet is smaller than the synthesis packet
- integration pass launched after all section reviews returned
- final report mentions the integration review

Now change one thing

In fixtures/sources.json replace the visual-arts TIMEOUT with [] and rerun: a valid empty result must not read as a failure.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.